Server Hardening

Adrianna Holden-Gouveia
Website: aholdengouveia.name
: aholdengouveia

Accessibility Notice

This document is also available in HTML format at:

Link: `https://aholdengouveia.name/LinuxAdmin/labexcercises/serverhardening.html`

The HTML version provides enhanced accessibility features including keyboard navigation, screen reader support, responsive design, dark mode support, and high contrast options.

Objectives:

- The objective of this lab is to introduce students to server hardening and security auditing. Students will run a security audit with Lynis, analyze and fix the issues it finds, and debug and refactor a poorly written health monitoring script. These skills help keep production servers secure and well monitored, and prepare students to work with code of varying quality.

Background

Acme Corp's report servers need a security audit, and the team's health check script needs work. The script was written by me using an AI. Scripts written by AI can be of varying quality, and it's likely you'll get slop and need to fix it. This script is slop on purpose and needs to be fixed. Be wary

of the comments by previous developers (me!), you never know who you can trust, here be dragons! You will be working on the two servers you set up in the Server Setup lab: your Red Hat based server and your second server.

Requirements

1. Download and install Lynis on both your servers and run it.
2. Create a short report on the findings (one report for each server) and what you'll do to improve your server setup.
3. Fix the BadServerHealthCheck script to monitor the health of your server, using the commands from the PowerPoints on Security, DFIR and Backups as your base. There is a bash version and a Python version, and you may pick which one you want to fix. There are some hints in the comments for things to fix. Not everything that should be fixed is hinted at, some things that need fixing are included and hidden for extra coding chaos. Think about what info you care about, and how to make it easier for you to read or upload to your dashboard. Data is only good if you're using it for something.
4. When fixing, debugging, and refactoring the script, add comments for what each thing actually does.

Acceptance Criteria

- **Everything must be cited.** Every command, statement, and line of code in your scripts and documents must be cited, either with a link to a reference or with a note naming the previous class where you learned it, for example: `# Learned in CIS 117 Intro to Linux`. In scripts, put the citation in a comment next to the code. Anything without a citation will not count.
- Lynis is run on both servers, with a findings report for each.
- The fixed script is tested on both servers.
- Scripts with no documentation and no commentary will not be accepted.
- The script is uploaded to GitHub.
- The script meets all of the Script Goals below.
- Every terminal screenshot shows your prompt with your username and

hostname, and either the output of `date` or a note beside the terminal with your name and the term. Hostnames must match your server inventory from the Server Setup lab.

- Your GitHub repository shows your work over time: several commits across more than one day, each with a message describing the change, not a single upload of finished code.

Script Goals

Your fixed script must meet all of these goals:

1. The report runs once each time the script is run.
2. There is no fake or random data in the report.
3. There are no pointless delays.
4. Every check produces correct results, and none of them error.
5. There is no unused code, and no leftover temp files. If you need a temp file, create it safely and delete it when you're done.
6. Each run produces one clean report, saved in a sensible location (not /tmp), without repeating old results.
7. The network check doesn't depend on a single outside host.
8. Thresholds for CPU, memory, and disk are set as variables at the top of the script, so they are easy to change.
9. All comments are professional and accurate.
10. ShellCheck (for bash) or flake8 (for Python) reports no errors.

Deliverables

Audience: a new intern at Acme Corp whose first set of jobs is to check the health of all our report servers.

1. Your Lynis reports, including any changes you made to each server and why you made those changes.
2. Your fixed health check script, well commented, with a link to your GitHub where you've uploaded it.
3. Health monitoring documentation, including:
   (a) A short explanation of what you chose to change on your health monitor and why.
   (b) The location where the health report is saved, and instructions for how to run your script remotely.

(c) What you picked and fixed, why you picked those commands, and how they are used.
(d) Answer the following: Are they different for each server or the same for both? Are the running instructions any different?

Resources

References, a video, a PowerPoint and some notes are available at my website Link: `https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html`

Lynis Link: `https://cisofy.com/lynis/`

The BadServerHealthCheck script, bash version Link: `https://www.aholdengouveia.name/LinuxAdmin/labexcercises/BadServerHealthCheck.sh`

ShellCheck, for checking bash scripts Link: `https://www.shellcheck.net/`

flake8, for checking Python scripts Link: `https://flake8.pycqa.org/`

The BadServerHealthCheck script, Python version Link: `https://www.aholdengouveia.name/LinuxAdmin/labexcercises/BadServerHealthCheck.py`